

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.16

TRANSACTIONS

 Copy as Markdown

Transactions

A transaction bundles several database operations into one unit that either fully succeeds or fully fails. Other concurrent operations never see the intermediate states. If any step fails, none of the steps affect the database.

Serverpod handles database transactions through the `session.db.transaction` method. The method takes a callback function that receives a transaction object.

The transaction is committed when the callback function returns, and rolled back if an exception is thrown. Any return value of the callback function is returned by the `transaction` method.

Pass the transaction object to each database operation method to include them in the same atomic operation:

```
var result = await session.db.transaction((transaction) async {  
  // Do some database queries here.  
  await Company.db.insertRow(session, company, transaction: transaction);  
  await Employee.db.insertRow(session, employee, transaction: transaction);  
  
  // Optionally return a value.  
  return true;  
});
```

In the example we insert a company and an employee in the same transaction. If any of the operations fail, the entire transaction will be rolled back and no changes will be made to the database. If the transaction is successful, the return value will be `true`.

TIP

The Serverpod test tools use this rollback behavior to isolate test cases: by default, each test runs in a transaction that is rolled back when the test ends. See [rollback configuration](#) in the testing docs.



Cancel a transaction

To roll back a transaction deliberately, without throwing an exception, call `cancel` on the transaction object:

```
var result = await session.db.transaction((transaction) async {
    await Company.db.insertRow(session, company, transaction: transaction);

    if (companyLimitReached) {
        await transaction.cancel();
        return false;
    }

    return true;
});
```

Cancelling rolls back all of the transaction's changes, while the `transaction` method still returns the callback's return value. Database calls made after the cancel have no effect and might throw, depending on the database driver.

Transaction isolation

The transaction isolation level can be configured when initiating a transaction. The isolation level determines how the transaction interacts with concurrent database operations. If no isolation level is supplied, the level is determined by the database engine.

INFO

The default isolation level for the Postgres database engine is `IsolationLevel.readCommitted`.

INFO

Setting an isolation level is only supported on Postgres. On SQLite, the `isolationLevel` setting is ignored, and transactions always run serialized.

To set the isolation level, configure the `isolationLevel` property of the `TransactionSettings` object:

```
await session.db.transaction(
    (transaction) async {
        await Company.db.insertRow(session, company, transaction: transaction);
        await Employee.db.insertRow(session, employee, transaction: transaction);
    },
    settings: TransactionSettings(isolationLevel: IsolationLevel.serializable),
);
```

In the example the isolation level is set to `IsolationLevel.serializable`.

The available isolation levels are:

Isolation Level	Constant	Description
Read uncommitted	<code>IsolationLevel.ReadUncommitted</code>	Exhibits the same behavior as <code>IsolationLevel.ReadCommitted</code> in Postgres
Read committed	<code>IsolationLevel.ReadCommitted</code>	Each statement in the transaction sees a snapshot of the database as of the beginning of that statement.
Repeatable read	<code>IsolationLevel.RepeatableRead</code>	The transaction only observes rows committed before the first statement in the transaction was executed giving a consistent view of the database. If any conflicting writes among concurrent transactions occur, an exception is thrown.
Serializable	<code>IsolationLevel.Serializable</code>	Gives the same guarantees as <code>IsolationLevel.RepeatableRead</code> but also throws if read rows are updated by other transactions.

For a detailed explanation of the different isolation levels, see the [PostgreSQL documentation](#).

Deferred constraint checking

Foreign keys are checked at the end of every statement, so a transaction that inserts a child row before its parent fails on the child insert. Mark the relation as `deferrable` and set `deferConstraints` to postpone the check until the transaction commits. Rows can then be written in any order, as long as every foreign key is satisfied at commit.

```
await session.db.transaction(
    (transaction) async {
        // The child Employee row is inserted before the parent Company row.
        await Employee.db.insertRow(session, employee, transaction: transaction);
        await Company.db.insertRow(session, company, transaction: transaction);
    },
    settings: TransactionSettings(deferConstraints: true),
);
```

Relations declared with `deferred` are already checked at commit and need no setting.

Transaction failure exceptions

When the database rejects a query inside the transaction, Serverpod throws a `DatabaseQueryException`. This can happen, for example, when concurrent writes conflict with the selected transaction isolation level, or when Postgres detects a deadlock.

The exact database error code depends on why Postgres rejected the query. Serverpod exposes Postgres error code constants through `PgErrorCode`, so you can compare them with the `code` field on `DatabaseQueryException`.

```
try {
  await session.db.transaction(
    (transaction) async {
      await Company.db.updateRow(
        session,
        company,
        transaction: transaction,
      );
    },
    settings: TransactionSettings(isolationLevel: IsolationLevel.serializable),
  );
} on DatabaseQueryException catch (e) {
  if (e.code == PgErrorCode.serializationFailure ||
      e.code == PgErrorCode.deadlockDetected) {
    // Retry the transaction or report a write conflict to the caller.
    return;
  }

  rethrow;
}
```

For all PostgreSQL error codes, see the [PostgreSQL error code appendix](#).

Savepoints

A savepoint is a special mark inside a transaction that allows all commands that are executed after it was established to be rolled back, restoring the transaction state to what it was at the time of the savepoint.

Read more about savepoints in the [PostgreSQL documentation](#).

Creating savepoints

To create a savepoint, call the `createSavepoint` method on the transaction object:

```
await session.db.transaction((transaction) async {
  await Company.db.insertRow(session, company, transaction: transaction);
  // Create savepoint
  var savepoint = await transaction.createSavepoint();
  await Employee.db.insertRow(session, employee, transaction: transaction);
});
```

```
});
```

In the example, we create a savepoint after inserting a company but before inserting the employee. This gives us the option to roll back to the savepoint and preserve the company insertion.

Rolling back to savepoints

Once a savepoint is created, you can roll back to it by calling the `rollback` method on the savepoint object:

```
await session.db.transaction((transaction) async {  
  // Changes preserved in the database  
  await Company.db.insertRow(session, company, transaction: transaction);  
  
  // Create savepoint  
  var savepoint = await transaction.createSavepoint();  
  
  await Employee.db.insertRow(session, employee, transaction: transaction);  
  // Changes rolled back  
  await savepoint.rollback();  
});
```

In the example, we create a savepoint after inserting a company. We then insert an employee but invoke a rollback to our savepoint. This results in the database preserving the company but not the employee insertion.

Releasing savepoints

Savepoints can also be released, which means that the changes made after the savepoint are preserved in the transaction. Releasing a savepoint will also render any subsequent savepoints invalid.

To release a savepoint, call the `release` method on the savepoint object:

```
await session.db.transaction((transaction) async {  
  // Create two savepoints  
  var savepoint = await transaction.createSavepoint();  
  var secondSavepoint = await transaction.createSavepoint();  
  
  await Company.db.insertRow(session, company, transaction: transaction);  
  await savepoint.release();  
});
```

In the example, two savepoints are created. After the company is inserted the first savepoint is released, which renders the second savepoint invalid. If the second savepoint is used to rollback, an exception will be thrown.